

LAB # 14

Implementation of Enum and Interface

Objective

Implement, understand and practice the concepts of Enum and Interface in Java.

Lab Goals

- Introduction to Enum and Interface
- How to implement Enum and Interface in java?
- **Understanding Enums:** Learn how to define and use enums in Java to represent a fixed set of constants.
- **Exploring Interface:** Gain familiarity with interfaces and their role in defining contracts for classes in Java.
- **Syntax Comprehension:** Understand the syntax for declaring enums and interfaces in Java.

Introduction to Enum

Enum, short for Enumeration, is a special data type that allows a programmer to define a set of constants. In Java, an enum is a data type that contains fixed set of constants.

Syntax:

```
enum EnumName {  
    CONSTANT1,  
    CONSTANT2,  
    // More constants can be added here  
}
```

Example Code#1:

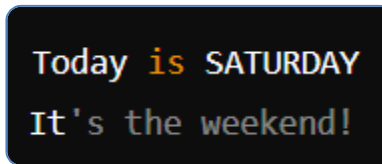
```
public class EnumExample {  
    enum Day {  
        SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,  
        SATURDAY  
    }  
    public static void main(String[] args) {  
        Day today = Day.SATURDAY;  
        System.out.println("Today is " + today);  
  
        switch (today) {  
            case MONDAY:  
                System.out.println("It's Monday!");  
                break;  
            case SATURDAY:
```

```

        case SUNDAY:
            System.out.println("It's the weekend!");
            break;
        default:
            System.out.println("It's a weekday!");
    }
}
}

```

Output:



```

Today is SATURDAY
It's the weekend!

```

Example Code#2: Enum with Constructor and Methods

```

enum Planet {
    MERCURY(3.303e+23, 2.4397e6),
    VENUS(4.869e+24, 6.0518e6),
    EARTH(5.976e+24, 6.37814e6);

    private final double mass; // in kilograms
    private final double radius; // in meters
    Planet(double mass, double radius) {
        this.mass = mass;
        this.radius = radius;
    }
    public double getMass() {
        return mass;
    }

    public double getRadius() {
        return radius;
    }
}

public class EnumExample {
    public static void main(String[] args) {
        for (Planet planet : Planet.values()) {
            System.out.println("Planet: " + planet + ", Mass: " + planet.getMass() + ", Radius: " +
planet.getRadius());
        }
    }
}

```

Output:

```
Planet: MERCURY, Mass: 3.303E23, Radius: 2439700.0
Planet: VENUS, Mass: 4.869E24, Radius: 6051800.0
Planet: EARTH, Mass: 5.976E24, Radius: 6378140.0
```

Introduction of Interface in Java

An interface in Java is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types. It cannot contain implementation of the methods. Interfaces form a contract between the class and the outside world, and this contract is enforced at compile time.

Syntax:

```
interface InterfaceName {
    // Constant declarations
    // Method signatures
}
// Interface can also extend other interfaces
interface InterfaceName extends SuperInterface {
    // Constant declarations
    // Method signatures
}
```

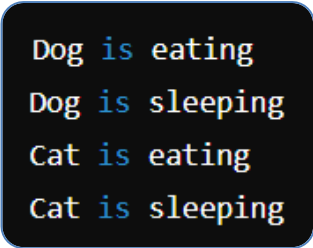
Example Code#1:

```
interface Animal {
    void eat();
    void sleep();
}
class Dog implements Animal {
    public void eat() {
        System.out.println("Dog is eating");
    }

    public void sleep() {
        System.out.println("Dog is sleeping");
    }
}
class Cat implements Animal {
    public void eat() {
        System.out.println("Cat is eating");
    }
}
```

```
        public void sleep() {
            System.out.println("Cat is sleeping");
        }
    }
    public class InterfaceExample {
        public static void main(String[] args) {
            Dog dog = new Dog();
            dog.eat();
            dog.sleep();

            Cat cat = new Cat();
            cat.eat();
            cat.sleep();
        }
    }
}
```

Output:

```
Dog is eating
Dog is sleeping
Cat is eating
Cat is sleeping
```

Example Code#2: Interface with Static Method

```
interface MathOperation {
    static int add(int a, int b) {
        return a + b;
    }
    static int subtract(int a, int b) {
        return a - b;
    }
}
public class InterfaceExample {
    public static void main(String[] args) {
        int resultAdd = MathOperation.add(5, 3);
        System.out.println("Addition: " + resultAdd); // Output: Addition: 8
        int resultSubtract = MathOperation.subtract(10, 4);
        System.out.println("Subtraction: " + resultSubtract); // Output: Subtraction: 6
    }
}
```

Output:

```
Addition: 8  
Subtraction: 6
```

Exercise

- A. Create an enum representing different types of coffee drinks i.e. **(ESPRESSO(2.5), LATTE(3.0), CAPPUCINO(3.5);)** with prices. Write a method to calculate the total cost of ordering a certain quantity of a specific coffee.
- B. Create an enum representing different pizza sizes i.e. **SMALL(8), MEDIUM(10), LARGE(12);** with their respective diameters. Write a method to find the largest and smallest pizza size from a given array of pizza sizes.
- C. Define an interface Playable with a method play(). Implement this interface for classes Guitar and Piano. Write a program to demonstrate the playability of a guitar and a piano.
- D. Define an interface BankAccount with methods deposit(double amount) and withdraw(double amount). Implement this interface for classes SavingsAccount and CheckingAccount. Additionally, each account should have a method getBalance() to retrieve the current balance. Write a program to demonstrate depositing, withdrawing, and checking balance for both a savings account and a checking account.
- E. Define an interface Shape with methods calculateArea() and calculatePerimeter(). Implement this interface for classes Circle, Rectangle, and Triangle. Each class should calculate and return the area and perimeter of its respective shape. Write a program to demonstrate calculating the area and perimeter of a circle, rectangle, and triangle.